🚀 Day 30 — React Components & Props

Lahore E-Commerce Product Card Grid

Date: January 14, 2026 (Tuesday) | **Phase 2, Week 5**

> 🎯 **Mission:** By end of today, you will build reusable product cards for a Lahore E-commerce app — understanding HOW React components think, not just how they look.

⚡ MAX OMEGA PRIME INTEGRITY CHECK

Before reading further — answer these in your head:

1. ✅ Was your Integrity Session completed today (7:30 AM – 10:30 AM)?
2. ✅ Do you have your Day 29 proof? (Counter App committed to GitHub?)
3. ✅ Can you explain in one sentence: *What is JSX?*

If any answer is NO — stop. Re-read Day 29 notes first. Aage jaana mana hai.

PART 1: CONCEPT FOUNDATION 🧠

Building the RIGHT Mental Model First

🏪 The Anarkali Market Analogy — Understanding Components

Imagine you're walking through **Anarkali Bazaar** in Lahore. You see hundreds of *dukaans* (shops). Now notice something interesting:

Every dukaan has the same STRUCTURE:

- 🪧 A sign with the shop name
- 📦 Products on display
- 💰 Prices shown
- 🧑 A shopkeeper inside

But **every dukaan has DIFFERENT DATA:**

- One sells shoes (Rs. 2,500)
- One sells fabric (Rs. 800/meter)
- One sells electronics (Rs. 45,000)

This is EXACTLY what React Components are.

> A **Component** is a reusable template (the dukaan structure).
> **Props** are the specific data that fills that template (the actual products, prices, names).

You design the dukaan structure ONCE. Then you "open" it 100 times with different data each time.

📱 The Daraz App Analogy — Why Components Matter

Open Daraz.pk in your mind. The homepage shows 50+ product cards. Now ask yourself:

> Did a developer write 50 different HTML blocks for 50 products?

NO! They wrote ONE `<ProductCard>` component and used it 50 times.

Without components:

```
// Old way — Vanilla JS / Plain HTML
<div class="card">Samsung TV - Rs. 85,000 ⭐⭐⭐⭐</div>
<div class="card">iPhone 14 - Rs. 320,000 ⭐⭐⭐⭐⭐</div>
<div class="card">Washing Machine - Rs. 45,000 ⭐⭐⭐</div>
// ... copy-paste 47 more times 😭
```

With components:

```
// React way — write ONCE, use EVERYWHERE 🥕
<ProductCard name="Samsung TV" price={85000} rating={4} />
<ProductCard name="iPhone 14" price={320000} rating={5} />
<ProductCard name="Washing Machine" price={45000} rating={3} />
```

That's the power. Write once. Reuse everywhere.

🧠 Pause & Think Moment #1

Before reading further, answer this in your own words:

> *"In Anarkali Market, what is the 'component' and what are the 'props'?"*

Write your answer on paper. Don't skip this — it builds real understanding.

🔄 Props vs State — The Critical Distinction

This is where many beginners get confused. Let's fix that RIGHT NOW.

The Marriage Analogy (Pakistani Context 😄)

Think about a **Nikah Certificate**:

- **Props** = Information written ON the certificate (groom's name, bride's name, date, location). This information was *given to* the certificate from outside. The certificate itself doesn't change this

info.

- **State** = The *current status* of the marriage (happy, complicated, divorced 😅). This can *change over time* based on what happens internally.

Concept	Props	State
Who controls it?	Parent component (outside)	The component itself (inside)
Can it change?	No — read-only inside component	Yes — changes trigger re-render
Where does it come from?	Passed from parent	Created inside component
Example	Product name, price	Is item in cart? Is modal open?

Simple Rule to Remember:

> 🔒 **Props** = Data FROM outside = Read-only = Like a printed pamphlet
> 🔄 **State** = Data INSIDE component = Can change = Like a live scoreboard

Today we focus 100% on Props. State comes in Day 31.

🏗️ Component Composition — The LEGO Analogy

You know how LEGO works? Small blocks combine to build big things.

React works the SAME way:

```
📦 App (the whole Lahore E-commerce site)
├── 🔝 Header (logo + search bar)
├── 📋 ProductGrid (the grid of all products)
│   ├── 🃏 ProductCard (single product — our focus today!)
│   ├── 🃏 ProductCard
│   └── 🃏 ProductCard
└── 🔻 Footer (links + copyright)
```

ProductCard is a small LEGO block.
ProductGrid combines multiple ProductCards.
App uses everything together.

You don't build the whole building at once. You build small blocks. Today = building the ProductCard block.

PART 2: FUNDAMENTAL BUILDING BLOCKS 🔨

One Concept at a Time

📌 Building Block #1 — What IS a Component?

A React component is simply a **JavaScript function that returns JSX**.

That's it. Don't overcomplicate it.

```
// This is a complete, valid React component
function Greeting() {
  return <h1>Assalam o Alaikum, Lahore!</h1>;
}
```

Three rules for components:

1. Name MUST start with Capital Letter (`Greeting`, not `greeting`)
2. Must return JSX (or null)
3. Can only return ONE root element

Common Mistake #1 — Returning multiple elements without wrapper:

```
// ❌ WRONG — Two root elements
function Card() {
  return (
    <h1>Title</h1>
    <p>Description</p>  // Error! Can't have two root elements
  );
}

// ✅ CORRECT — Wrap in a div (or use Fragment <>)
function Card() {
  return (
    <div>
      <h1>Title</h1>
      <p>Description</p>
    </div>
  );
}
```

📌 Building Block #2 — Passing Props (The Parcel Analogy)

Think of props like sending a **TCS parcel** from Lahore to Islamabad:

- You (parent component) pack the parcel with items (props)
- You address it to the recipient (component name)
- The recipient opens the parcel and uses the items

```jsx
// PARENT packs and sends the parcel
function App() {
  return (
    <ProductCard
      name="Samsung TV"        {/* Item 1 in parcel */}
      price={85000}            {/* Item 2 in parcel */}
      rating={4}               {/* Item 3 in parcel */}
    />
  );
}

// CHILD receives and opens the parcel
function ProductCard(props) {
  // props is the parcel — it contains everything sent
  console.log(props);
  // { name: "Samsung TV", price: 85000, rating: 4 }

  return (
    <div>
      <h2>{props.name}</h2>
      <p>Rs. {props.price}</p>
    </div>
  );
}
```

Important: Notice `price={85000}` uses `{}` not quotes. Numbers and variables go in `{}`. Strings can use `""` or `{}`.

📌 Building Block #3 — Destructuring Props (Cleaner Code)

Instead of writing `props.name`, `props.price` everywhere, we can destructure:

```jsx
// 🔴 Without destructuring (verbose)
function ProductCard(props) {
  return <h2>{props.name} — Rs. {props.price}</h2>;
}

// ✅ With destructuring (clean & professional)
function ProductCard({ name, price, rating }) {
  return <h2>{name} — Rs. {price}</h2>;
}
```

Both work exactly the same. But destructuring is how real developers write React.

📌 Building Block #4 — Conditional Rendering

This is where your Day 23-24 JavaScript skills come back!

The `inStock` prop needs to show different UI based on its value:

```
// Using ternary operator for conditional rendering
function ProductCard({ name, price, inStock }) {
  return (
    <div>
      <h2>{name}</h2>
      <p>Rs. {price}</p>

      {/* Ternary: condition ? ifTrue : ifFalse */}
      {inStock
        ? <span style={{color: 'green'}}>✅ In Stock</span>
        : <span style={{color: 'red'}}>❌ Out of Stock</span>
      }
    </div>
  );
}
```

Using && for "show or show nothing":

```
{/* Only show rating if it exists */}
{rating && <p>⭐ {rating}/5</p>}
```

📌 Building Block #5 — Using a Component Multiple Times

```
function App() {
  return (
    <div>
      {/* Same component, different data — magic! */}
      <ProductCard name="Samsung TV" price={85000} rating={4} inStock={true}
/>
      <ProductCard name="iPhone 14" price={320000} rating={5} inStock={true}
/>
      <ProductCard name="Headphones" price={8500} rating={3} inStock={false}
/>
    </div>
  );
}
```

Each `<ProductCard>` is an independent instance with its own data. Change one, others are unaffected.

⚠️ Common Mistakes — Learn From Others' Pain

Mistake 1: Lowercase component name

```
function productCard() { ... }  // ❌ React thinks it's an HTML tag
function ProductCard() { ... }  // ✅ React knows it's a component
```

Mistake 2: Modifying props inside component

```
function ProductCard({ price }) {
  price = price * 1.17;  // ❌ NEVER modify props — they're read-only!

  const priceWithTax = price * 1.17;  // ✅ Create new variable instead
}
```

Mistake 3: Forgetting curly braces for expressions

```
<h2>props.name</h2>       // ❌ Shows literal text "props.name"
<h2>{props.name}</h2>     // ✅ Shows actual value like "Samsung TV"
```

PART 3: PROGRESSIVE LEARNING PATH 📈

I Do → We Do → You Do

🎯 Stage 1 — I DO: Study This Working Example

Read this carefully. Understand EVERY line before moving on.

```
// ============================================
// FILE: ProductCard.jsx
// PURPOSE: Reusable card component for products
// ============================================

function ProductCard({ name, price, rating, inStock }) {
  // Destructure 4 props from parent

  // Helper: Convert number to star string
  // If rating=4, creates "⭐⭐⭐⭐"
  const stars = '⭐'.repeat(rating);

  return (
    <div className="product-card">
```

```
      {/* Product name */}
      <h3>{name}</h3>

      {/* Price formatted with commas for Pakistani Rupees */}
      <p className="price">Rs. {price.toLocaleString()}</p>

      {/* Rating using our stars helper */}
      <p className="rating">{stars} ({rating}/5)</p>

      {/* Conditional: Different UI for stock status */}
      {inStock
        ? <button className="btn-add">Add to Cart 🛒</button>
        : <button className="btn-disabled" disabled>Out of Stock</button>
      }

    </div>
  );
}

// ============================================
// FILE: App.jsx
// PURPOSE: Parent that uses ProductCard
// ============================================

function App() {
  return (
    <div className="product-grid">
      <h1>🛍️ Lahore Electronics</h1>

      <ProductCard
        name="Samsung 55\" 4K TV"
        price={95000}
        rating={4}
        inStock={true}
      />

      <ProductCard
        name="JBL Bluetooth Speaker"
        price={12500}
        rating={5}
        inStock={false}
      />

    </div>
  );
}
```

🧠 **Pause & Think Moment #2:**

- *Why does `price.toLocaleString()` work here? What does it do?*
- *What happens if you pass `inStock={false}` to the second card?*

- *Why is the component named `ProductCard` and not `productcard`?*

Answer all three before moving to Stage 2.

🎯 Stage 2 — WE DO: Complete the Gaps Together

Below is a component with `TODO` gaps. Fill them using what you've learned.

Task: Build a `ReviewCard` component for customer reviews

```
function ReviewCard({ customerName, city, reviewText, stars, verified }) {

  // TODO 1: Create a variable 'starEmojis' that repeats '⭐' based on
'stars'
  const starEmojis = _____;

  return (
    <div className="review-card">

      {/* TODO 2: Display the customerName in an <h4> tag */}
      _____

      {/* TODO 3: Display city in a <p> tag with text "From: {city}" */}
      _____

      {/* TODO 4: Display starEmojis in a paragraph */}
      <p>{_____}</p>

      {/* TODO 5: Show reviewText inside quotes */}
      <p>"{reviewText}"</p>

      {/* TODO 6: Only show "✅ Verified Purchase" if verified is true */}
      {_____ && <span>✅ Verified Purchase</span>}

    </div>
  );
}

// Test your component with this data:
<ReviewCard
  customerName="Ahmed Raza"
  city="Lahore"
  reviewText="Bohat acha product hai! Highly recommend."
  stars={4}
  verified={true}
/>
```

Hints (read only if stuck for 5+ minutes):

- Hint 1: `'⭐'.repeat(n)` repeats a string n times

- Hint 2: JSX elements look like `<tag>content</tag>`
- Hint 3: For conditional rendering with `&&`, think: `{variable && <element />}`

🎯 Stage 3 — YOU DO: Independent Practice Problems

🟢 Tier 1 — Basic (5-10 minutes)

Build a `CityBadge` component that receives:

- `city` (string) — e.g., "Lahore"
- `province` (string) — e.g., "Punjab"
- `population` (number) — e.g., 13000000

It should display:

```
🏙️ Lahore
Punjab, Pakistan
Population: 13,000,000
```

Success check: Does `population.toLocaleString()` format the number with commas?

🟡 Tier 2 — Intermediate (10-15 minutes)

Build a `FoodOrderCard` component for a Foodpanda-style app:

Props: `restaurantName`, `dish`, `price`, `deliveryTime`, `isAvailable`

Requirements:

- Show restaurant name as heading
- Show dish name and price
- Show "🕒 25-30 mins" using deliveryTime prop
- If `isAvailable` is true: show "Order Now 🍕" button
- If `isAvailable` is false: show "Currently Unavailable" in red text

Use it 3 times with different data in your App.jsx

🔴 Tier 3 — Challenge (15-20 minutes)

Build a `StudentResultCard` component for a Pakistani university:

Props: `studentName`, `rollNo`, `marks` (out of 100), `subject`

Requirements:

- Show student name, roll number, subject
- Calculate percentage and display it

- Based on marks, show grade AND background color:
  - 80+ = "A" grade, green background
  - 60-79 = "B" grade, yellow background
  - 40-59 = "C" grade, orange background
  - Below 40 = "Fail", red background

Thinking hint: You'll need an if/else or switch statement INSIDE the component to determine grade. Define this logic in a variable before the return statement.

🐛 Debugging Challenge

This code has 4 bugs. Find them WITHOUT running the code first:

```
function menuitem({ itemname, price, isSpicy }) {
  return (
    <div>
      <h3>itemname</h3>
      <p>Price: Rs. price</p>
      isSpicy && <span>🌶️ Spicy!</span>
      <button onclick={() => alert('Added!')}>Add</button>
    </div>
    <p>Menu Item</p>
  );
}
```

Bugs to find:

1. Bug related to component naming
2. Bug where values show as literal text
3. Bug with JSX structure
4. Bug with event handler name (remember: React uses camelCase)

Write down all 4 bugs and their fixes before checking against the solution at bottom of this document.

PART 4: INDEPENDENT APPLICATION — PROJECT 🏆

Lahore E-Commerce Product Card Grid

🎯 Your Mission

Build a **Lahore Electronics E-commerce product grid** with reusable components.

You are NOT given the complete solution. You must build this yourself.

📋 Project Requirements

Files to Create:

- `src/components/ProductCard.jsx` — The reusable card
- `src/components/ProductGrid.jsx` — Grid that holds multiple cards
- `src/App.jsx` — Main app using the grid
- `src/App.css` — Styling

ProductCard Props (must support all):

Prop	Type	Example
`name`	string	"Samsung 55" 4K TV"
`price`	number	95000
`image`	string	URL to image
`rating`	number (1-5)	4
`inStock`	boolean	true
`brand`	string	"Samsung"

Required Features:

1. ✅ **ProductCard component** — displays all 6 props
2. ✅ **Star rating display** — show actual ⭐ emojis based on rating number
3. ✅ **In Stock / Out of Stock** — different visual treatment
4. ✅ **Price formatting** — use `.toLocaleString()` for Pakistani Rupees
5. ✅ **ProductGrid component** — renders an array of ProductCard components
6. ✅ **Minimum 6 products** — use real Lahore electronics store prices (do research!)
7. ✅ **Conditional styling** — Out of Stock cards should look different (grey out, disabled button)

🗺️ Implementation Roadmap (Build in This Order!)

Milestone 1 — Single Card Working (30 minutes)

- Create ProductCard with hardcoded data first
- Get it rendering correctly in App.jsx
- Then switch hardcoded data to props

Milestone 2 — Multiple Cards Working (20 minutes)

- Use ProductCard 3 times manually in App.jsx
- Verify each card shows different data

Milestone 3 — ProductGrid Component (20 minutes)

- Create a `products` array with 6 product objects
- Build ProductGrid that maps over the array
- Each object in array becomes props for a ProductCard

Milestone 4 — Stock Conditional (15 minutes)

- Ensure inStock=true shows "Add to Cart" button
- Ensure inStock=false shows disabled/greyed state
- Test both cases

Milestone 5 — Polish & Style (remaining time)

- Make it look like a real e-commerce site
- Responsive grid layout
- Hover effects on cards

💡 Thinking Frameworks (Use When Stuck)

Stuck on how to map products array?

```
Think: I have an array of objects.
Each object needs to become a <ProductCard>.
I know how to use .map() from Week 2!
array.map(item => <Component key={item.id} {...item} />)
```

Stuck on spread operator `{...item}`?

```
{...item} = "take all properties of item object and pass as individual props"
It's like: <ProductCard name={item.name} price={item.price} rating=
{item.rating} />
But shorter!
```

Stuck on the key prop?

```
React needs 'key' when rendering lists.
Use a unique ID from your data.
key helps React track which items changed.
```

✅ Self-Assessment Checklist

Before calling project "done", verify:

- ☐ I can explain what a component is to a 10-year-old

- [ ] I can explain the difference between props and state
- [ ] ProductCard component works with any product data I pass
- [ ] Changing `inStock` to false changes the UI correctly
- [ ] I used destructuring in my component (not `props.name`)
- [ ] My component files start with Capital Letter
- [ ] No prop is being modified inside the component
- [ ] Code is committed to GitHub with clear commit message

🤔 Apply to New Context Questions

1. Could this same `ProductCard` component work for a clothing store? What props would you change?
2. What if you had a `PremiumProductCard` that was fancier? How would you share common code between them?
3. If you had 500 products (like Daraz), would you still manually write 500 `<ProductCard>` components? What's the better approach?

🚨 ORAL GATEKEEPING EXAM PREP

MAX OMEGA PRIME Will Ask You These

Level 1 — Remember (Concepts)

- What is a React component?
- What are props and who controls them?
- Why must component names start with a capital letter?

Level 2 — Understand (Explain)

- Explain props vs state using the Anarkali Market analogy in your own words
- Why would you use conditional rendering for `inStock`?
- What does destructuring props mean and why do we do it?

Level 3 — Apply (Code Challenge — 10 minutes)

Build a `CareemDriverCard` component with these props:

- `driverName`, `rating`, `tripsCompleted`, `isOnline`
- Show driver info
- Show "🟢 Available" if online, "🔴 Offline" if not
- Only show "⭐ Top Driver" badge if rating >= 4.5

Level 4 — Analyze

- What would break if you changed a prop value inside a component?

- Why is it better to have many small components than one giant component?

Level 5 — Teach Back

"Explain React components and props as if teaching your younger brother who knows basic HTML but has never seen React."

🐛 Debugging Challenge — Answer Key

(Read ONLY after you've attempted to find bugs yourself)

```
Bug 1: 'menuitem' → should be 'MenuItem' (Capital letter required)
Bug 2: <h3>itemname</h3> → <h3>{itemname}</h3> (Need curly braces)
         <p>Price: Rs. price</p> → <p>Price: Rs. {price}</p>
Bug 3: Two root elements (div + p) → wrap in a single parent div
Bug 4: onclick → onClick (React uses camelCase event handlers)
```

📈 Tomorrow Preview — Day 31

Topic: State with `useState` Hook

You'll take today's ProductCard and add:

- ❤️ Wishlist toggle (added/not added)
- 🛒 Cart counter
- 📊 Live quantity selector

Today's components will "come alive" with state. The mental model you built today (Props = data from outside) is the foundation. Tomorrow: State = data that changes inside.

Tonight's prep: Think about this question:

> *"In your ProductCard, what information might CHANGE while the user is on the page? That's where state lives."*

💪 MAX OMEGA PRIME's Closing Message

Sharjeel, sun! Aaj tu ne ek bahut important concept seekha — **component-based thinking**. Yeh sirf React ki baat nahi hai. Yeh **professional developer ki soch** hai — har cheez ko reusable, modular, aur focused rakhna.

Jab kal Devsinc ka HR tera portfolio dekhega, woh nahi dekhe ga ke tune kitni jaldi seekha. Woh dekhega ke tera **code organized hai ya nahi**. Reusable components likhna — that's organized code.

Aaj ka kaam complete kar. GitHub commit karna mat bhoolo. Aur kal 7:30 AM pe fresh ho ke aana — useState ke saath yeh cards "zinda" ho jaenge! 🔥

Ab ja — laptop khol, code likh, aur deliver kar!

Day 30 Complete — January 14, 2026 | MAX OMEGA PRIME Approved